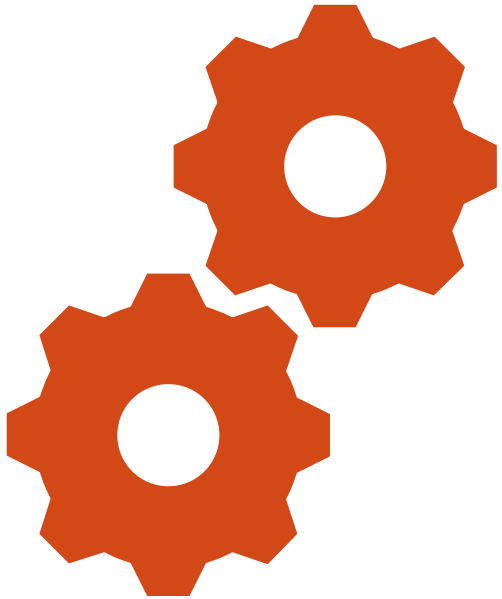


Mastering Firmware Testing: From Unit Testing to CI/CD with Code Coverage

BHARATH G

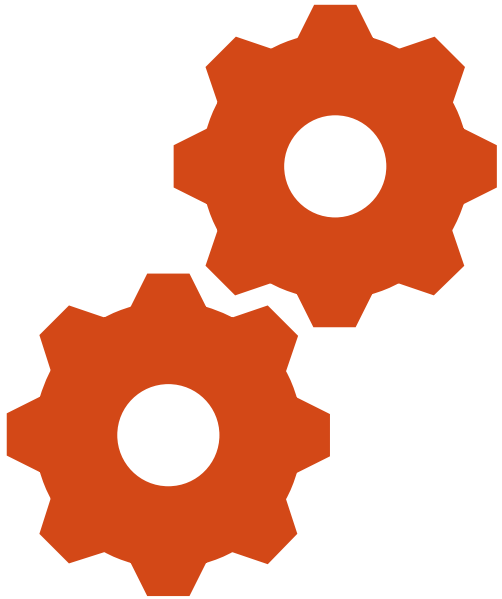
Staff Firmware Engineer | Dexcom

- Contributor to EFY forum since 2022
- 10+ Years of Experience in Firmware Development
- Medical, Aerospace and IoT device Design
- Embedded C++ | RTOS | Bluetooth Low Energy | Linux
- Technical Training | Career Guidance

STIJO JOSEPH

Senior Firmware Engineer | Dexcom

- Spent 4+ years building high-reliability Embedded systems
- Medical and Healthcare applications.
- Embedded C++ | RTOS | Python | Embedded Linux
- Current contributing to Continuous Glucose Monitoring Technologies.
- Focused on Intersection of Hardware/Software
- Contributes to end to end Product Development Life Cycle.

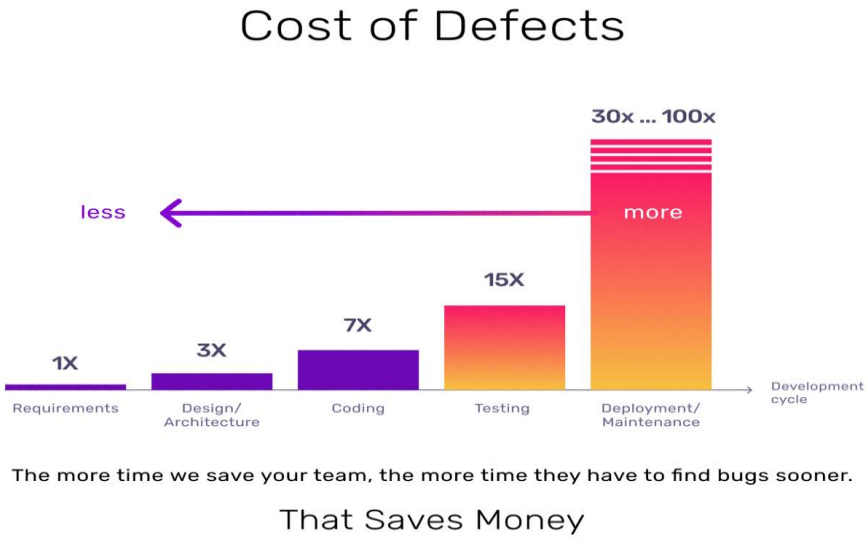


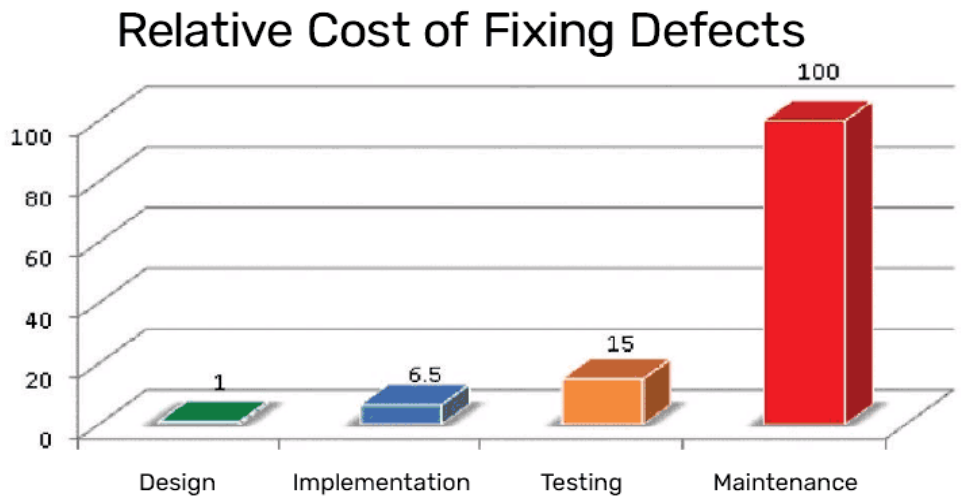
INTRODUCTION TO UNIT TESTING

COST OF FIXING BUGS IN SDLC

- In many embedded projects, testing is often deferred until hardware is available.
- But by then, debugging becomes expensive, slow, and dependent on limited resources.
- But most bugs don't come from hardware, they come from logic.

Unit testing changes this completely.





Source: The Cost of Finding Bugs Later in the SDLC and IBM System Science Institute Relative Cost of Fixing Defects

WHAT IS UNIT TESTING?

- Isolation of a section of code and validating its correctness.
- Testing each modules behaviour.
- Involves breaking a program into pieces and subjecting each piece to a series of tests.

IMPORTANCE OF UNIT TESTING

Unit testing helps to:

- Verify code correctness and Improves code quality
- Improve code confidence before integration
- Enable faster debugging
- Save development time and cost by detecting bugs early in development
- Enable faster integration and supports safe refactoring and reuse
- Enable code coverage measurement and detect dead code
- Acts as living documentation

WHY UNIT TESTING MATTERS IN EMBEDDED ?

- Embedded software is tightly coupled with hardware, but not all of it needs hardware to be validated.
- A significant portion of firmware includes:
  - Data processing logic
  - State machines
  - Protocol handling
  - Error handling etc.
- These can and should be tested independently.

HOW TO DO UNIT TESTING?

By following the AAA method:

- Arrange (Setup the test conditions):
  - Initialize inputs and preconditions required for the test.
  - This isolates the unit under test and ensures a controlled environment.
- Act (Execute the behaviour):
  - Call the function or method being tested.
  - This step should contain only the action under test and no additional logic.
- Assert (Verify the outcome):
  - Check that the result matches the expected behaviour using assertions.
  - This validates correctness, including outputs, state changes, or interactions.

MISCONCEPTIONS AND TRUTHS

- **Unit testing takes too much time**

→ Truth: It may add effort upfront, but significantly reduces debugging and rework time later.

- **Unit testing will find all bugs**

→ Truth: It won't catch everything. It helps build reliable components and reduce defects early.

- **100% code coverage means fully tested code**

→ Truth: High coverage doesn't guarantee correctness. Quality of test cases matters more than numbers.

ROLE OF SIMULATORS IN UNIT TESTING

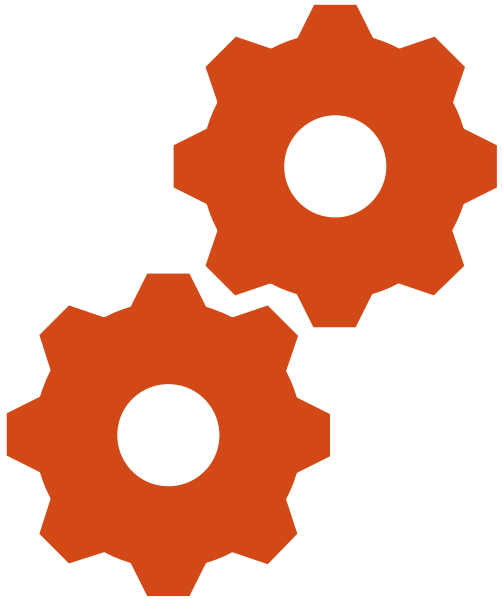
Simulators enable developers to test embedded code without requiring actual hardware

- Makes development faster and more flexible.
- They allow controlled testing of edge cases, fault conditions, and rare scenarios that are difficult or risky to reproduce on real devices.
- By isolating software from hardware dependencies, simulators:
  - Improve test repeatability
  - Enable early validation in the development cycle
  - Support automation in CI pipelines.

- Open-Source Embedded System Simulators are highlighted in this post: 3 Best Open-Source Embedded System Simulators

SUMMARY

- Unit testing in embedded systems becomes truly effective when combined with structured test patterns, lightweight frameworks, and simulator or Hardware-in-loop based validation.
- By integrating unit testing framework tools, developers can validate functionality early, isolate defects quickly, and maintain reliable code across iterations.
- A disciplined approach:
  - writing unit tests alongside code, using simulators, and integrating into CI ensures scalable, maintainable, and production ready embedded software.



HANDS-ON LABS

LAB 1 – UNIT TESTING C CODE

Agenda

- Need of Frameworks and available Frameworks
- Create a function
- Write Unit tests
- Compile and Run
- Understand the output
- Understand Unit test framework

UNIT TEST FRAMEWORK

- Unit test framework is not mandatory
  - We can use Asserts to do the testing
  - Requires additional logic to print summary and other details to console/terminal
- Framework provides everything needed for AAA.
- Most of the Unit test Frameworks are based on xUnit
- Best Frameworks for Embedded software
  - CppUTest
  - CUnit
  - Unity Throw The Switch
  - Google Test
  - Proprietary – Parasoft, Cantata etc.
- List of unit testing frameworks - Wikipedia

LAB SETUP

- C Compiler (GCC or any IDE)
- **Unity Test Framework source code** (Download .c and .h files from the src folder)
- A sample C function (Logic isolated from hardware).

WRITE THE FUNCTION - CODE TO BE TESTED

Create a header and source code files

checksum.h

```
#ifndef CHECKSUM_H
#define CHECKSUM_H
#include <stdint.h>
/* The checksum is computed by summing all the bytes in the buffer
and taking the result modulo 256. */
uint8_t calculate_checksum(uint8_t *buffer, uint8_t length);
#endif
```

checksum.c

```
#include "checksum.h"
uint8_t calculate_checksum(uint8_t *buffer, uint8_t buffer_length)
{
    uint16_t result_checksum = 0;
    uint8_t loop_index = 0;
    for (loop_index = 0; loop_index < buffer_length; loop_index ++)
    {
        result_checksum += buffer[loop_index ];
    }
    return (uint8_t)(result_checksum % 256);
}
```

WRITE THE TEST CASES

Create a test source code file

Test_checksum.c

```
#include "unity.h"
#include "checksum.h"
void setUp(void) { }
void tearDown(void) { }

void test_calculate_checksum_with_valid_data(void) {
    uint8_t data[] = {0x01, 0x02, 0x03, 0x04};
    uint8_t expected_checksum = 10;
    uint8_t actual_checksum = calculate_checksum(data, sizeof(data));
    TEST_ASSERT_EQUAL_UINT8(expected_checksum, actual_checksum);
}
void test_calculate_checksum_with_empty_data(void) {
    uint8_t data[] = {};
    uint8_t expected_checksum = 0;
    uint8_t actual_checksum = calculate_checksum(data, sizeof(data));
    TEST_ASSERT_EQUAL_UINT8(expected_checksum, actual_checksum);
}
int main(void) {
    UNITY_BEGIN();
    RUN_TEST(test_calculate_checksum_with_valid_data);
    RUN_TEST(test_calculate_checksum_with_empty_data);
  return UNITY_END();
}
```

BUILD, RUN AND OBSERVE OUTPUT

Build:

- `gcc checksum.c unity.c test_checksum.c -o Lab1`

Run

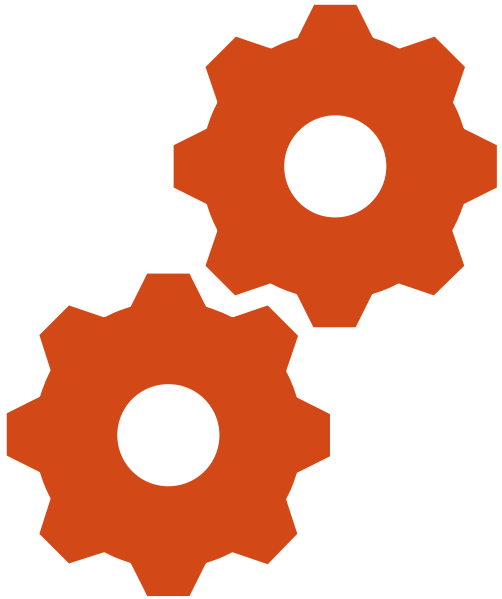
- `./Lab1`

Expected Results

```
test_checksum.c:39:test_calculate_checksum_with_valid_data:PASS
test_checksum.c:41:test_calculate_checksum_with_max_values:PASS
-----------------------
2 Tests 0 Failures 0 Ignored
OK
```

SUMMARY OF LAB1

- Code should be written as functions to perform Unit testing
- Use of Test Framework makes Unit testing easier
- Each framework has its own APIs, but the functionality is same.



LAB - 2

LAB 2 – CODE COVERAGE ANALYSIS

Agenda

- Importance of Code coverage
- Types of Code coverage
- Compile for Coverage
- Generate Coverage Report
- Methods to Improve Coverage
- Gcov vs Lcov

LAB SETUP

- Lab1 source code
  - Checksum.c, checksum.h, test_checksum.c
- GCC Compiler
- gcov (comes with GCC)
- lcov (for report generation)

IMPORTANCE OF CODE COVERAGE

- Code Coverage Addresses the question:
  - *How much of our code is actually tested?*
- Helps identify:
  - Untested code paths
  - Missing edge cases
  - Dead or redundant code
  - Identifies gaps in code early
- Helps to ensure:
  - Critical logic is exercised
  - Edge conditions are validated
  - Provides confidence before integration

TYPES OF CODE COVERAGE

- **Function Coverage**
  - Were all functions tested?
- **Line Coverage**
  - Were all lines tested?
- **Branch Coverage**
  - Were all decision paths tested?

Which of the above is most important ?

GENERATE COVERAGE REPORT

Build

```
gcc -fprofile-arcs -ftest-coverage \
        checksum.c test_checksum.c unity.c -o Lab2
```

Run

```
./Lab2
```

This generates coverage data files (.gcda, .gcno).

Expected Results

```
File 'checksum.c'
Lines executed:100.00% of 8
Creating 'checksum.c.gcov'

Lines executed:100.00% of 8
```

HOW TO IMPROVE COVERAGE

Add additional test cases

- To test all Branches
- To test all possible Decision paths
- To test all loops and iterations
- For Invalid inputs and parameters
- For Boundary checks
- For Edge conditions

For your code to be robust

- It must handle all the invalid conditions, edge cases, negative scenarios etc. and return known results without crashing.

GCOV VS LCOV

GCOV

- Generates low level coverage data directly from GCC
- Helps you see which lines of code were executed during tests.

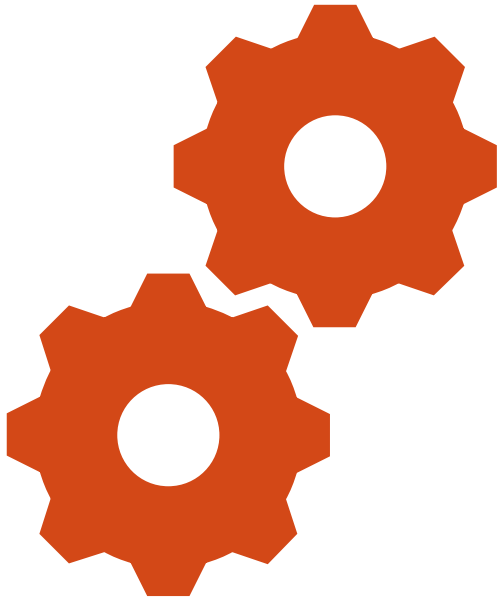
LCOV

- Builds on gcov
- Provides clean, visual HTML reports
- Makes it easier to analyse and share coverage info.

SUMMARY OF LAB2

- Code coverage gives visibility into what your tests actually validate.
- Writing tests is not enough, we need to measure coverage.
- Focus on branch coverage, not just lines.

Coverage only highlights gaps, but doesn't guarantee correctness.



LAB - 3

LAB 3 – CODE COVERAGE ON RENODE

Agenda

- Renode as simulator for STM32
- Unit testing on Renode
- Code Coverage on Renode
- Renode in CI Pipeline

LAB 3 – UNIT TESTING ON STM32 FIRMWARE USING RENODE

- In the previous labs, we:
  - Wrote unit tests for a function.
  - Did Code Coverage Analysis.
  - All on Host machine.
- Can we do all of this on MCU?
  - Yes, we can!
  - Locally and in CI pipeline.

LAB SETUP

- Lab2 source code
  - Checksum.c, checksum.h, test_checksum.c
- Build tools
  - gcc-arm-none-eabi
  - binutils-arm-none-eabi
- Unity test framework
  - Unity Throw the switch (source files only)

SM32F4 SPECIFIC CODE

- Startup file containing function
  - `Reset_Handler and Interrupt_Handler`
  - `Interrupt vector Table`

- Linker script containing Program Memory Map

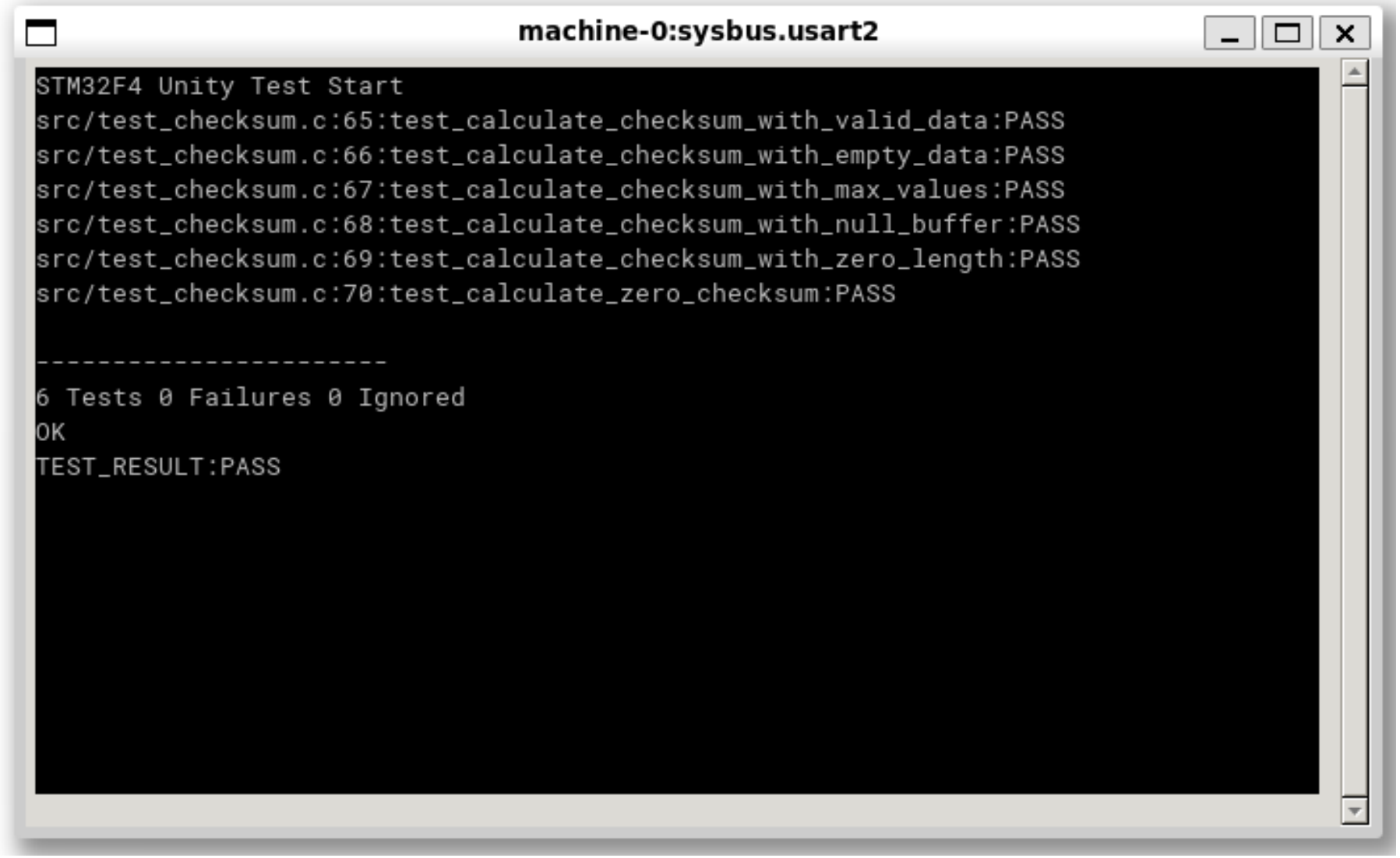
- To print unit test results on UART terminal, create function definitions for:
  - `void uart_putc(char c);`
  - `void uart_print(const char *s);`
  - `int putchar(int c);`

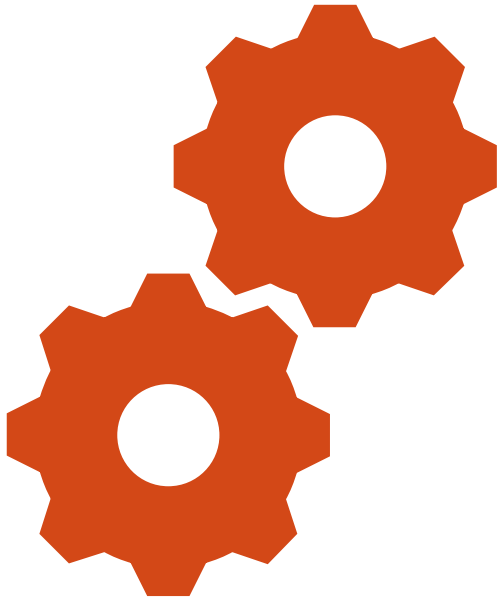
BUILD THE FIRMWARE WITH FLAGS

```
CC = arm-none-eabi-gcc

CFLAGS = -mcpu=cortex-m4 -mthumb \
         -O0 -g \
         -ffreestanding -fno-builtin\
         -Isrc -Iunity -Istm32f4 \
         -Wall -Wextra

LDFLAGS = -T stm32f4/linker.ld \
          -nostartfiles \
          -Wl,--gc-sections
```

RUN THE ELF ON RENODE

RESULTS


machine-0:sysbus.usart2
STM32F4 Unity Test Start
src/test_checksum.c:65:test_calculate_checksum_with_valid_data:PASS
src/test_checksum.c:66:test_calculate_checksum_with_empty_data:PASS
src/test_checksum.c:67:test_calculate_checksum_with_max_values:PASS
src/test_checksum.c:68:test_calculate_checksum_with_null_buffer:PASS
src/test_checksum.c:69:test_calculate_checksum_with_zero_length:PASS
src/test_checksum.c:70:test_calculate_zero_checksum:PASS
----
6 Tests 0 Failures 0 Ignored
OK
TEST_RESULT:PASS



COVERAGE ON RENODE

COVERAGE OF FIRMWARE ON RENODE

Add coverage specific instructions to renode script

stm32.resc

```
mach create
machine LoadPlatformDescription @platforms/boards/stm32f4_discovery-kit.repl
sysbus LoadELF @LAB3.elf
sysbus.cpu CreateExecutionTracing "tracer" @./coverage/trace.bin PC truesysbus.cpu
EnableOpcodesCounting true
showAnalyzer sysbus.usart2
sysbus.usart2 AddLineHook "TEST_RESULT:" "#print 'completed'";sysbus.cpu
GetAllOpcodesCounters;quit
start
```

- **Build the firmware code as earlier and run renode simulation**
- **Then, run renode execution tracer**

```
renode --disable-xwt renode/stm32.resc

python3 /opt/renode/tools/execution_tracer/execution_tracer_reader.py \
        coverage ./coverage/trace.bin \
        --binary Lab3.elf \
        --sources ./src/checksum.c
```

COVERAGE OF FIRMWARE ON RENODE

Example output from Renode execution tracer:

```
TN:
    SF:./src/checksum.c
    DA:5,5
    DA:6,5
    DA:7,3
    DA:10,2
    DA:11,2
    DA:13,10
    DA:15,8
    DA:17,2
    DA:18,5
    end_of_record
```

Save this output to ***coverage.info*** file for further analysis

This is an Address cache which tells which lines ran, how many times

`DA:5,5 means line 5 ran 5 times.`

EXTRACT COVERAGE FROM COVERAGE.INFO

Method1: Run lcov to get % of coverage

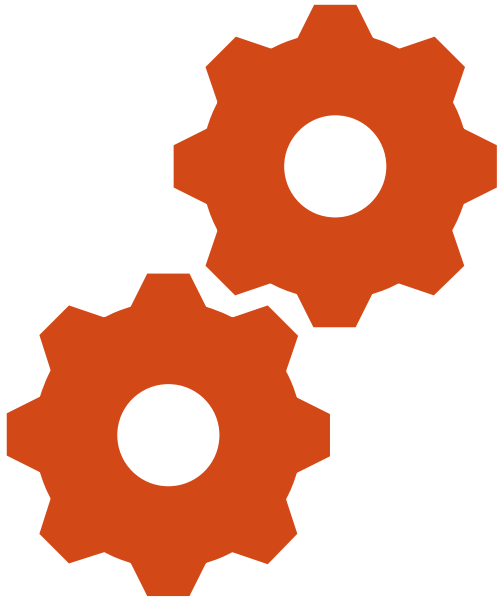
```
lcov --summary coverage/test.info
```

Method2: Use awk command to extract % of coverage.

```
cat coverage/test.info | awk -F',' '/DA:/ {found++; if($2>0)
hit++} END {printf "Coverage: %.2f%% (%d/%d)\n",
(hit/found)*100, hit, found}'
```

Both of the above commands give % of coverage of code.

```
Total Coverage: 100.00% (9/9 lines hit)
```



RENODE IN CI PIPELINE

UNIT TESTING WITH RENODE IN CI PIPELINE

Create a renode script

stm32.resc

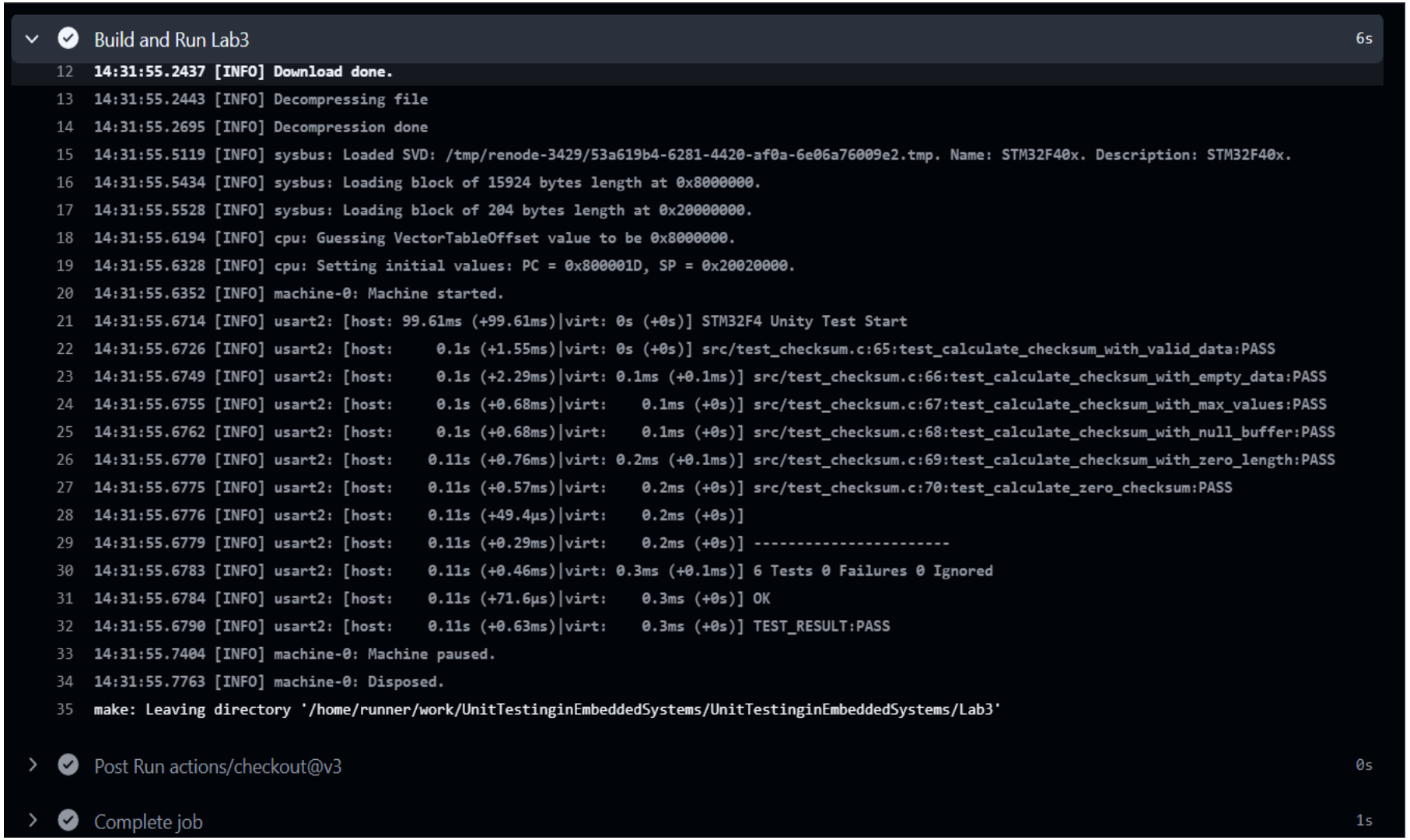
```
mach create
machine LoadPlatformDescription @platforms/boards/stm32f4_discovery-kit.repl
sysbus LoadELF @LAB3.elf
showAnalyzer sysbus.usart2
sysbus.usart2 AddLineHook "TEST_RESULT:" "#print 'completed'";quit
start
```

Add targets in Actions file to install Renode and run renode script

Build.yml

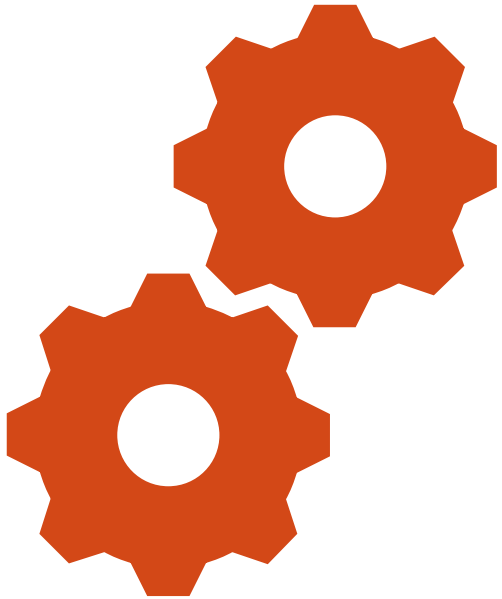
```
- name: Install Renode
      run: |
         wget
https://github.com/renode/renode/releases/download/v1.16.1/renode_1.16.1_amd64.deb
         sudo dpkg -i renode_1.16.1_amd64.deb
- name: Compile Firmware
      run: make -C Lab3
- name: Run tests
      run: renode --disable-xwt stm32.resc
```

GITHUB ACTIONS RESULTS



SUMMARY OF LAB 3

- Run Firmware without Hardware
- UART-based automated testing in headless simulation
- Renode enables Unit testing and Coverage measurement in CI pipeline.



CONCLUSION

FROM WRITING FIRMWARE TO ENGINEERING RELIABLE FIRMWARE

Lab 1 — Unit Testing Fundamentals

- Writing testable embedded C code
- Assertions and structured test cases using Unity

Lab 2 — Code Coverage Analysis

- Measuring test effectiveness using gcov/lcov
- Identifying untested and dead code paths

Lab 3 — Embedded CI/CD with Renode

- Running firmware without hardware
- CI pipeline integration using GitHub Actions

KEY TAKEAWAYS

- Test firmware early - don’t wait for hardware
- Isolate logic from hardware dependencies
- Coverage shows what was executed, not what is well tested
- Simulation enables scalable and repeatable validation
- CI/CD transforms testing from manual activity to engineering workflow

Final Message

“Reliable firmware is not achieved by debugging more. It is achieved by validating earlier.”

Q & A

STAY CONNECTED

- Bharathgopal75@gmail.com
- Linkedin.com/in/Bharathgopal
- LinkedIn- QR Code